

torch.nn.functional.ctc_loss#

https://pytorch.org/docs/stable/generated/torch.nn.functional.ctc_loss.html

Description:

Compute the Connectionist Temporal Classification loss. See `CTCLoss` for details. In some circumstances when given tensors on a CUDA device and using CuDNN, this operator may select a nondeterministic algorithm to increase performance. If this is undesirable, you can try to make the operation deterministic (potentially at a performance cost) by setting `torch.backends.cudnn.deterministic = True`. See [Reproducibility](#) for more information.

Function Signature

```
torch.nn.functional.ctc_loss(log_probs, targets,  
input_lengths, target_lengths, blank=0, reduction='mean',  
zero_infinity=False)[source]#
```

Parameters

Return type

Returns:

Tensor

Code Examples

```
>>> log_probs = torch.randn(50, 16,
20).log_softmax(2).detach().requires_grad_()
>>> targets = torch.randint(1, 20, (16, 30), dtype=torch.long)
>>> input_lengths = torch.full((16,), 50, dtype=torch.long)
>>> target_lengths = torch.randint(10, 30, (16,),
dtype=torch.long)
>>> loss = F.ctc_loss(log_probs, targets, input_lengths,
target_lengths)
>>> loss.backward()
```

Notes & Warnings

NOTE

Note In some circumstances when given tensors on a CUDA device and using CuDNN, this operator may select a nondeterministic algorithm to increase performance. If this is undesirable, you can try to make the operation deterministic (potentially at a performance cost) by setting `torch.backends.cudnn.deterministic = True`. See Reproducibility for more information.

NOTE

Note This operation may produce nondeterministic gradients when given tensors on a CUDA device. See Reproducibility for more information.

Detailed Documentation

Documentation

Compute the Connectionist Temporal Classification loss.

See `CTCLoss` for details.

In some circumstances when given tensors on a CUDA device and using CuDNN, this operator may select a nondeterministic algorithm to increase performance. If this is undesirable, you can try to make the operation deterministic (potentially at a performance cost) by setting `torch.backends.cudnn.deterministic = True`. See Reproducibility for more information.

This operation may produce nondeterministic gradients when given tensors on a CUDA device. See Reproducibility for more information.

`log_probs` (Tensor) – (T,N,C)(T, N, C)(T,N,C) or (T,C)(T, C)(T,C) where C = number of characters in alphabet including blank, T = input length, and N = batch size. The logarithmized probabilities of the outputs (e.g. obtained with `torch.nn.functional.log_softmax()`).

`targets` (Tensor) – (N,S)(N, S)(N,S) or (sum(target_lengths)). May be an empty tensor if all entries in `target_lengths` are zero. In the second form, the targets are assumed to be concatenated.

`input_lengths` (Tensor) – (N)(N)(N) or `()()`. Lengths of the inputs (must each be $\leq T \leq T$)

`target_lengths` (Tensor) – (N)(N)(N) or `()()`. Lengths of the targets

`blank` (int, optional) – Blank label. Default 000.

`reduction` (str, optional) – Specifies the reduction to apply to the output: 'none' | 'mean' | 'sum'. 'none': no reduction will be applied, 'mean': the output losses will be divided by the target lengths and then the mean over the batch is taken, 'sum': the output will be summed. Default: 'mean'

`zero_infinity` (bool, optional) – Whether to zero infinite losses and the associated gradients. Default: False Infinite losses mainly occur when the inputs are too short to be aligned to the targets.